Neural Network Parameter Estimation for Discrete Bayesian Networks with Applications in Medical Diagnosis

Kaylin Aspoas[a], Ritesh Ajoodha *(Supervisor)*[a] and Aashna Gaogle *(Co-supervisor)*[a]

[a]*School of Computer Science and Applied Mathematics, University of the Witwatersrand, Johannesburg, South Africa*

ARTICLE INFO

Keywords:
Bayesian networks
Parameter estimation
Neural networks
Medical diagnosis

ABSTRACT

Bayesian networks offer an interpretable framework well-suited to medical diagnosis, but their standard parameter estimation methods estimate a separate distribution for every configuration of a variable's parents, and become unreliable when data is scarce relative to the number of configurations. This research replaces the conditional probability table (CPT) at each node with a neural conditional probability distribution (neural CPD), a small neural network that shares parameters across parent configurations while preserving the interpretable graph structure. On synthetic networks whose CPTs share no structure across configurations, a smoothed CPT matched to the generating prior is optimal by construction; neural CPDs, which are not told that prior, stay within about two and a half times its divergence and beat unsmoothed maximum likelihood in nearly every setting. When the synthetic CPTs do share structure, neural CPDs beat even a validation-tuned smoothed CPT once nodes have six or more parents, with up to 4.4 times lower divergence. On a real clinical dataset with expert-elicited structures, neural CPDs outperform a validation-tuned smoothed CPT in held-out log-likelihood in all 108 cross-validated settings, and the smoothed CPT needs 3 to 67 times as much data to match them. They also predict diabetes far more accurately, because the diabetes node has the largest CPT in every expert graph.

1. Introduction

Predictive models are increasingly used to support medical decision making, which raises a difficult question: who is to blame when they get it wrong? In medicine, a diagnosis must be something that can be explained, questioned and justified. If a model contributes to a missed diagnosis and there is no reasoning to inspect, responsibility becomes difficult to assign, and a clinician has no way of knowing whether the model learned a medically meaningful relationship or an accidental pattern. Yet such models remain valuable, because diagnosis depends on many interacting factors, such as symptoms, test results, history and lifestyle, and a well-designed model can find patterns that would otherwise be missed. The aim is therefore to support clinical judgement in a way that remains understandable and accountable.

Bayesian networks offer a promising way to do this, because they represent medical reasoning as a visible structure of relationships that shows how each piece of information contributes to a diagnosis. However, traditional Bayesian networks struggle when these relationships become complex. Each variable's relationship to its causes is learned as a separate probability for every combination of those causes, and as the number of combinations grows, the data available to estimate each one shrinks. Deep learning models capture complex patterns but are hard to interpret, while Bayesian networks are easy to understand but limited by how they learn. This research investigates whether neural networks can be used inside the interpretable structure of a Bayesian network to combine these strengths.

The contribution of this work is threefold. First, it confirms under controlled conditions that the failure of tabular estimators such as Maximum Likelihood Estimation (MLE) is governed by the number of training rows per parent configuration, which falls as in-degree or cardinality increases, or as sample size decreases. Second, it evaluates neural conditional probability distributions (CPDs) as a drop-in replacement for conditional probability tables (CPTs) on synthetic networks. Without being told how the networks were generated, they stay within about two and a half times the divergence of the optimal smoothed CPT and comfortably beat unsmoothed MLE, and when parent configurations share structure they outperform even a validation-tuned smoothed CPT at high in-degree. Third, on a real clinical dataset with expert-elicited structures, they outperform a validation-tuned smoothed CPT in every setting tested, matching it with 3 to 67 times less data, and substantially improve the diagnosis of diabetes.

Section 2 reviews related work, Section 3 describes the methodology, Section 4 presents the results, and Section 5 concludes and outlines future work.

2. Related Work

2.1. Tabular Parameter Estimation

Parameter learning in Bayesian networks with a known DAG reduces to estimating node-wise conditional probability distributions independently. In discrete BNs, MLE estimates each CPT entry as a ratio of observed counts (Koller and Friedman, 2009, Sec. 17.2),

$$\hat{P}(X_i = x_k \mid \mathrm{Pa}(X_i) = u_j) = \frac{N_{ijk}}{N_{ij}}, \tag{1}$$

where N_{ijk} counts the samples in which $X_i = x_k$ and $\mathrm{Pa}(X_i) = u_j$. This works well when data is abundant, but zero-valued parameters from unobserved configurations propagate through inference and suppress otherwise valid

conclusions (Oniśko and Drużdżel, 2013). Because a variable with k parents of cardinality r has r^k configurations, even moderate in-degree produces tables too large to estimate from finite data (Oniśko, Drużdżel and Wasyluk, 2001; Koller and Friedman, 2009, Sec. 5.2).

Bayesian parameter estimation (BPE) with Dirichlet priors ensures non-zero probabilities (Koller and Friedman, 2009, Secs. 17.3–17.4). The strength of the prior, such as the equivalent sample size of the BDeu prior (Heckerman, Geiger and Chickering, 1995), must still be chosen, and informative priors are impractical to specify at the scale of clinical networks (Kyrimi, McLachlan, Dube, Neves, Fahmi and Fenton, 2021). Constraint-based priors such as CaMAP improve on uniform Dirichlet priors but require expert-provided parameter constraints (Di, Wang, He and Guo, 2021). Whatever the prior, each parent configuration is still estimated from its own counts, so a configuration that is rarely or never observed is left close to the prior.

2.2. Structured Parameter Estimation

Structured estimators reduce the number of free parameters by letting parent configurations share them. Noisy-OR gates make the number of parameters linear in the number of parents for binary variables, at the cost of independence assumptions that rarely hold exactly (Oniśko et al., 2001). Local structure, such as decision trees or default tables inside a CPT, ties together configurations that behave alike and gives more robust estimates from small samples (Friedman and Goldszmidt, 1996). Hierarchical Dirichlet processes smooth each configuration's estimate towards those of configurations that share some of its parent values, which makes Bayesian network classifiers competitive with random forests (Petitjean, Buntine, Webb and Zaidi, 2018).

2.3. Neural Networks in Probabilistic Models

Each of these methods imposes a particular form of sharing: an independence assumption, a tree over parent values, or a hierarchy over them. Parametric models instead learn the conditional distribution as a function of the parents, and the simplest neural CPD has no hidden layer. Sigmoid belief networks make each binary node a logistic function of its parents (Neal, 1992), and Rijmen (2008) replaced each CPT of a discrete network with a multinomial logistic regression on its parents; logistic and other generalised linear CPDs are now standard textbook models (Koller and Friedman, 2009, Ch. 5). A multinomial logistic CPD is the linear softmax used as a control in this report: it shares parameters across all configurations, but its log-probabilities are additive in the parents unless interaction terms are specified by hand, which grows the model back towards the full table.

A hidden layer removes the need to specify interactions in advance. Bengio and Bengio (2000) and the neural autoregressive distribution estimator (Larochelle and Murray, 2011) model high-dimensional discrete data as a product of conditionals computed by networks with a hidden layer, but they condition each variable on all earlier variables in an ordering rather than estimating the CPDs of a given graph. Per-variable networks on a DAG appear in causal discovery: Ke, Bilaniuk, Goyal, Bauer, Larochelle, Schölkopf, Mozer, Pal and Bengio (2019) model each categorical variable with a one-hidden-layer network whose one-hot inputs are masked by a hypothesised graph and whose softmax output gives the conditional distribution, but they use it to learn the graph from interventions rather than to estimate parameters on a fixed one. Closest to this report, Gaogle (2026) replaced the linear-Gaussian CPDs of Gaussian BNs with neural networks and found them competitive with classical baselines, including ridge regression, but the Gaussian setting assumes continuous, jointly normal variables, whereas clinical variables such as symptom presence and categorical risk levels are discrete.

2.4. Bayesian Networks in Medical Diagnosis

The estimation problem is especially pressing in medicine, where BNs are widely applied to diagnosis and risk prediction because their graphical structure makes probabilistic reasoning visible and inspectable (Kyrimi et al., 2021; Zhong, Yang, Sun, Wang, Yu, Nie, Liu, Xu, Zhang, Zhang, Qi, Ji, Liu, Zhao, Jiang, Li, Song, Yu, Yang, Yu, Feng, Guo, Yang and Xue, 2024). Interpretability is a requirement for safe clinical decision making (Hulsen, 2023), and post-hoc explanations of black-box models are criticised as unfaithful to the model's reasoning (Rudin, 2019). The structure of a clinical network is often elicited from experts rather than learned: Zahoor, Constantinou, Curtis and Hasanuzzaman (2025) elicited diabetes DAGs from domain experts at three levels of confidence, which this report uses as fixed structures. Clinical variables are also often collapsed into a few coarse categories, as in the published discretisation of these diabetes data (Zahoor et al., 2025), which keeps the CPTs small enough to estimate but discards information the experts' graph could otherwise use.

No previous work evaluates neural CPDs as a replacement for the CPTs of a fixed, expert-given discrete network, against smoothed tabular estimators whose prior is chosen on validation data. This report does so. Its neural CPD maps the one-hot-encoded parents through a hidden layer to a softmax output, sharing parameters across parent configurations while preserving the DAG factorisation, and the linear softmax isolates what the hidden layer adds.

3. Methodology

3.1. Problem Formulation

A Bayesian network is defined by a directed acyclic graph G, where each node has an associated conditional probability distribution, and the joint distribution factorises according to G as

$$P(X_1, \ldots, X_n) = \prod_{i=1}^{n} P(X_i \mid \mathrm{Pa}_{X_i}),$$

where Pa_{X_i} denotes the parents of X_i in G (Koller, Friedman, Getoor and Taskar, 2007).

Tabular estimators such as MLE and BPE estimate a separate distribution for every parent configuration. A variable

with k parents of cardinality r has r^k parent configurations, so with N training rows each configuration is observed on average ρ times, where

$$\rho = \frac{N}{r^k}. \tag{2}$$

Increasing in-degree or cardinality, or reducing the sample size, all reduce ρ in the same way. Once ρ falls near or below one, most configurations are seen rarely or never, and tabular estimates fall back on their defaults. Clinical networks often reach this regime, because diagnoses depend on many interacting factors while clinical data does not grow to match. The aim is a parameter estimator that remains reliable when ρ is small while keeping the graph, and so the interpretability, of the BN.

The evaluation has two parts. On synthetic networks with known parameters, estimators are compared under controlled in-degree, cardinality and sample size. On a real clinical dataset with expert-elicited structures, the same estimators are compared under three discretisations, which vary ρ through cardinality with the graph held fixed.

3.2. Estimators

MLE estimates each CPT with Eq. (1). A parent configuration that never appears in the data is assigned a uniform distribution, and a child state never observed under an observed configuration keeps a probability of zero.

BPE avoids these zeros by adding a symmetric Dirichlet pseudocount a to every cell before normalising, $\hat{P} = (N_{ijk} + a)/(N_{ij} + ar)$. The Dirichlet estimator uses $a = 0.5$, the same value as the α that generates the synthetic CPTs (Section 3.3). Because every synthetic CPT column is an independent $\mathrm{Dir}(0.5, \ldots, 0.5)$ draw, this estimator is the exact posterior mean of each column and minimises expected KL divergence from the ground truth. On synthetic data it is therefore a *matched-prior reference*, the best achievable by any estimator given the data, rather than a competitor that neural CPDs are expected to beat. The Laplace estimator uses $a = 1$, a standard default that does not depend on the generator, and shows how much of the reference's advantage comes from knowing α. Where no fixed pseudocount is optimal, on the structured ground truths and on real data, the pseudocount is chosen on validation data.

A neural CPD replaces the CPT of each node X_i with a network f_θ that maps the parents to a distribution over the states of X_i,

$$P(X_i \mid \mathrm{Pa}(X_i)) = f_\theta(\mathrm{Pa}(X_i)).$$

Each parent is one-hot encoded at its own cardinality and the encodings are concatenated, so the input width grows with the *sum* of parent cardinalities rather than their product. A root node receives a constant input and learns its marginal through the output bias. Hidden layers use ReLU activations, and a softmax output gives a valid distribution (Goodfellow, Bengio and Courville, 2016, Sec. 6.2.2.3). One network is trained per node by minimising the cross-entropy

$$\mathcal{L}(\theta) = -\frac{1}{N} \sum_{s=1}^{N} \log P_\theta\big(x_i^{(s)} \mid \mathrm{pa}_i^{(s)}\big),$$

where $x_i^{(s)}$ and $\mathrm{pa}_i^{(s)}$ are the observed state and parent configuration in row s (Goodfellow et al., 2016, Sec. 6.2.1.1). Training uses Adam (Kingma and Ba, 2015) for the linear softmax and AdamW (Loshchilov and Hutter, 2019) for the other networks. A share of the N rows is held out for early stopping (30% on synthetic data, 20% on real data), and each node stops independently on its validation loss. The tabular estimators fit on all N rows, so the networks see fewer rows, not more.

Table 1
Estimators. All networks use ReLU hidden layers and per-node early stopping; settings for each network and cardinality are in Supplementary Table S2.

Estimator	Description
MLE	CPT; uniform for unseen configurations
Dirichlet	CPT, pseudocount 0.5
Laplace	CPT, pseudocount 1
Dirichlet (tuned)	CPT, pseudocount chosen on validation data
`nn_linear`	Linear softmax (no hidden layer), untuned
`nn_32`	One hidden layer of 32 units
Tuned MLPs	One or two hidden layers of 32–256 units

Table 1 lists the estimators. The linear softmax has no hidden layer, so it is a multinomial logistic regression fitted per node: it shares parameters across configurations but cannot represent interactions between parents, and acts as a control for the effect of the hidden layer. It is run at untuned default settings. The other networks were configured separately for each cardinality on tuning DAGs that are disjoint from the evaluation DAGs, using only validation loss. An initial run without early stopping showed that larger networks overfit quickly, so every network uses early stopping. Bayesian hyperparameter sweeps at in-degrees 2, 6 and 10 then selected six architectures at $r = 2$ and five at $r = 4$, with one or two hidden layers of 32 to 256 units; in both sweeps the best configurations had 256 units or two hidden layers. The tuning results and every network's settings are given in the supplementary material (Supplementary Figures S1–S2 and Tables S1–S2). The main comparison uses `nn_32`, a single hidden layer of 32 units. Its KL divergence is on average 17% above that of the best tuned network at $r = 2$ and 8% above at $r = 4$, while it holds 2 to 66 times fewer parameters than the tuned architectures at $k = 10$ in 20-node networks.

3.3. Synthetic Experiments

Synthetic networks have known parameters, so the learned distributions can be compared directly with the true ones, which is not possible on real data. The DAGs are generated to control in-degree directly. The n nodes are placed in a random order, and the node at position i is given $\min(k, i)$

Table 2
Synthetic experimental grid. Every setting is crossed in full for every estimator: 500 configurations at r=2 and 300 at r=4, which uses the values marked †.

Setting	Values
Network size n	$10^{\dagger}$, $20^{\dagger}$, $30^{\dagger}$, 40, 50
Parent in-degree k	$2^{\dagger}$, $4^{\dagger}$, $6^{\dagger}$, $8^{\dagger}$, $10^{\dagger}$
Dirichlet α	$0.5^{\dagger}$
Training rows N	$200^{\dagger}$, $600^{\dagger}$, $1000^{\dagger}$, $1400^{\dagger}$
Cardinality r	2, $4^{\dagger}$
DAGs per setting	5
Test sample (KL only)	10 000 rows

parents, chosen uniformly from the nodes before it, so every node past the first k has in-degree exactly k.

Every column of every ground-truth CPT is drawn independently from a symmetric Dirichlet distribution, $P^*(X_i \mid \text{pa}) \sim \text{Dir}(\alpha, \dots, \alpha)$, with $\alpha = 0.5$. Because the columns are independent, the rows observed under one parent configuration carry no information about any other. Sharing parameters across configurations therefore cannot help in expectation, and the best an estimator can do is the matched-prior reference. These networks test how close an estimator comes to that optimum without being told the prior.

To test whether neural CPDs gain when configurations do share structure, a second family of ground truths uses the same 20-node DAGs at $r = 2$, with the same sample sizes and seeds. Each CPT is logistic: the score of each child state is the sum of a bias, one effect per parent and, optionally, effects for a few random pairs of parents, and the CPT column is the softmax of the scores. Configurations that agree on a parent therefore share that parent's effect. Two levels are used: purely additive CPDs, and CPDs in which half of the score variance comes from pairwise interactions. The score scale is set so that the columns are, on average, as sharp as Dirichlet(0.5) draws.

Table 2 lists the settings, which are crossed in full for every estimator, giving 500 configurations at $r = 2$ and 300 at $r = 4$. The number of training rows is held fixed as in-degree increases, so that any change in performance reflects the size of the conditional probability space rather than the amount of data. Training sets are drawn by ancestral sampling and nested, so the smaller sets are the first N rows of the largest. A separate test sample of 10 000 rows from each ground truth is used only to estimate KL divergence. Every random step has its own seed derived from the configuration, so all estimators see the same networks and data.

3.4. Real Clinical Data

The real-data experiments use the CDC Diabetes Health Indicators dataset (Centers for Disease Control and Prevention, 2017), drawn from the 2015 Behavioral Risk Factor Surveillance System (BRFSS): 253 680 survey responses across 21 health-related variables and a binary diabetes indicator. The structures are the expert-elicited diabetes DAGs of Zahoor et al. (2025) at three levels of expert confidence (Supplementary Figure S3). The high-confidence graph has 37 edges and a maximum in-degree of 7, the moderate-confidence graph has 50 edges and a maximum in-degree of 8, and the low-confidence graph has 55 edges and a maximum in-degree of 12. Only the DAG comes from the experts; all parameters are learned.

Every variable except body mass index is natively categorical, so the discretisations follow the survey instrument. The coarsest level (L1) reproduces the collapsed categories of Zahoor et al. (2025) and matches their published marginals to within one percentage point. L2 uses the categories as collected, for example 13 age bands and 8 income bands, and L3 bins body mass index into deciles and the day-count variables more finely. The decile cut points are computed from each training fold alone, and are the same in every fold. Across the nine combinations of discretisation and DAG, the CPTs hold from 1 348 free parameters (L1, high confidence) to over 9 million (L3, low confidence).

Estimators are compared by three-fold cross-validation, stratified on diabetes status. Within each training fold, nested subsets of $N \in \{500, 2000, 10\,000, 50\,000\}$ rows are drawn, and the whole held-out fold is the test set. The networks are `nn_32` and `nn_128_64` with their $r = 4$ settings, together with the untuned linear softmax. Because the real-data subsets are far larger than the tuning sets, the batch size grows in proportion to N by a rule fixed before any run, so that the number of optimisation steps per epoch stays close to that at tuning.

Two hyperparameters are tuned by nested cross-validation: each network's learning rate (3×10^{-4}, 10^{-3} or 3×10^{-3}) and the Dirichlet pseudocount (0.1, 0.5 or 1.0). Within each outer fold, each candidate is scored by its log-likelihood on the validation 20% of each subset, averaged over all 36 combinations of discretisation, DAG and N, and the best is selected separately in each fold. A single value per fold is used because one combination's validation set can be as small as 100 rows, and because a pseudocount encodes a prior belief that should not depend on N. The selected candidate is refitted with the same rows and scored once on that fold's test set, which plays no part in any choice. Every fold selected a learning rate of 3×10^{-3} for both networks and a pseudocount of 1.0, in each case the largest value in its grid.

3.5. Evaluation

On synthetic data, the metric is the Kullback-Leibler divergence from the ground-truth joint distribution P^* to the learned distribution $\hat{P}$ (Koller and Friedman, 2009, App. A.1.3). The sum over all r^n joint states is intractable, so the chain-rule decomposition is used, with a Monte Carlo estimate over the $S = 10\,000$ test rows $x^{(s)} \sim P^*$:

$$D_{\text{KL}}(P^* \| \hat{P}) = \mathbb{E}_{P^*}\Big[\sum_i D_{\text{KL}}\big(P_i^* \parallel \hat{P}_i\big)\Big] \approx \frac{1}{S}\sum_{s=1}^{S}\sum_{i=1}^{n}\sum_{x_i} P_{is}^*(x_i)\log\frac{P_{is}^*(x_i)}{\hat{P}_{is}(x_i)},$$

where $P^*_{is}(x_i) = P^*(x_i \mid \mathrm{pa}_i^{(s)})$ is the true distribution of X_i given its parents in row s, and $\hat{P}_{is}$ is the learned one. Only the parent configurations are sampled; the sum over each child's states is exact. Because KL is summed over nodes it grows with network size, so results that pool settings use a single network size (30 nodes).

Where MLE assigns zero probability to a child state with positive true probability, the divergence is infinite, so such zeros are floored at $\epsilon = 10^{-12}$, as in other evaluations of unsmoothed estimators (Nafar, Venable, Cui and Kordjamshidi, 2026); the same floor applies to MLE on the real data. Most of MLE's divergence then comes from these floored cells, so its size depends on ϵ, and $\epsilon = 10^{-6}$ roughly halves it. MLE is therefore reported as a reference for the cost of not smoothing, and quantitative comparisons rest on the smoothed estimators, which never assign zero probability, as in Niculescu, Mitchell and Rao (2006).

On real data the ground truth is unknown, so performance is measured by the mean held-out log-likelihood per row, $\frac{1}{S}\sum_s \sum_i \log \hat{P}(x_i^{(s)} \mid \mathrm{pa}_i^{(s)})$. Its expected negative is the cross-entropy $H(P^*, \hat{P}) = H(P^*) + D_{\mathrm{KL}}(P^* \,\|\, \hat{P})$, and $H(P^*)$ does not depend on the estimator, so differences in log-likelihood are exactly differences in KL divergence. The unseen-configuration rate, the fraction of test rows in which at least one node's parent configuration never occurs in training, is also reported. For diagnosis, the probability of diabetes given every other variable is computed exactly from the Markov blanket of the diabetes node, its own CPD and that of its single child, and scored by the area under the ROC curve (AUROC), the Brier score and the log-loss.

Free parameters are counted as $(r-1)$ times the number of configurations for a CPT, and every weight and bias for a network. Training time is recorded in separate serial runs, since parallel execution inflates wall-clock times: on synthetic data for one 20-node network at in-degrees 2, 6 and 10 with $N \in \{600, 1400\}$, and on real data for three settings spanning the size of the CPTs.

4. Results

4.1. How Tabular Methods Fail

The synthetic setup reproduces the well-known behaviour of tabular estimators under sparse data (Oniśko et al., 2001; Koller and Friedman, 2009, Sec. 17.2). Figure 1 shows that KL divergence and CPT memory both rise sharply once a node's parent configurations (2^k) outnumber the training rows. In 40-node networks, MLE's KL divergence rises from 0.28 to 58 nats and its memory from 2.5 to 508 KB as in-degree goes from 2 to 10. The Dirichlet estimator removes the zero probabilities and has about fifteen times lower error at $k = 10$, but follows the same pattern, rising from 0.07 to 3.81 nats.

The cause is data coverage rather than computation. In 20-node networks at $N = 1000$, the share of parent configurations never seen in training rises from 0.5% at $k = 2$ to 78% at $k = 10$, while every MLE fit takes under one second. Both tabular estimators assign a uniform distribution

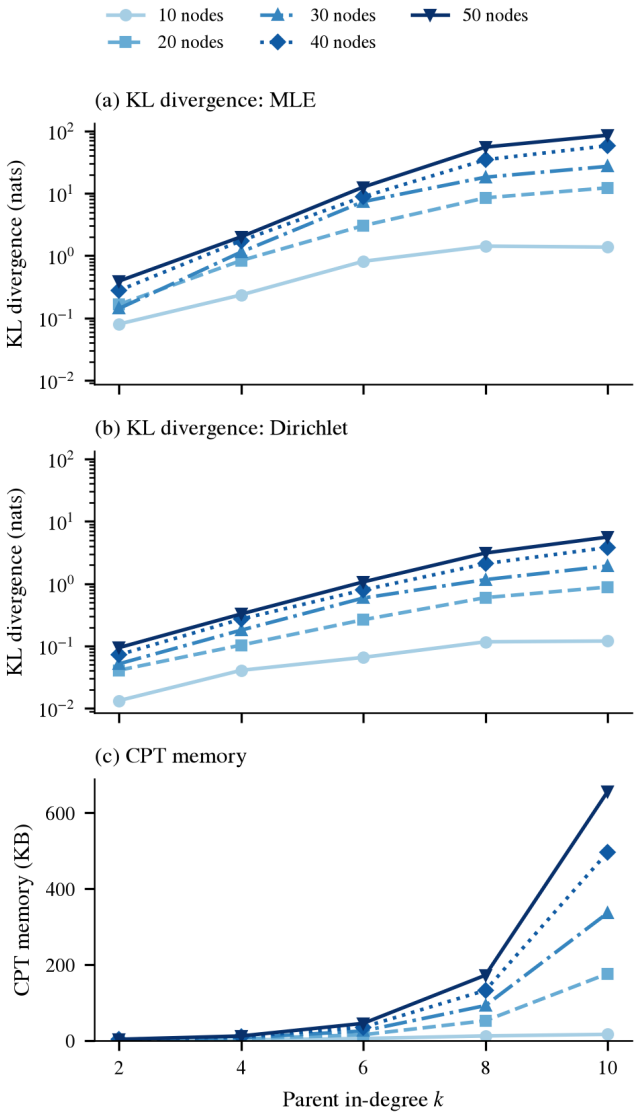


Figure 1: Tabular estimators on synthetic networks ($r = 2$) as in-degree increases ($N = 1000$, $\alpha = 0.5$, mean over five DAGs; darker is larger). KL divergence to the ground truth for (a) MLE and (b) the Dirichlet estimator (shared log-scaled y-axis). (c) Total CPT memory, the same for both.

to every unseen configuration, so smoothing removes zero probabilities but cannot recover these columns.

4.2. Neural CPD Architectures

Figure 2 compares every neural architecture with the tabular estimators at $r = 2$. Every MLP has lower KL divergence than MLE at every in-degree, and the gap widens as the configuration space grows, while all of them stay above the matched-prior Dirichlet reference. The $r = 4$ grid shows the same ordering. The MLPs differ little from one another, and no single architecture is best everywhere: at $r = 2$, `nn_256` is the best tuned network in 237 of the 500 configurations and `nn_128_64` in 97.

The linear softmax behaves differently. At $r = 2$ it beats MLE in only 8% of configurations at $k = 2$, and overtakes it only once the configuration space is large (72% at $k = 6$ and 98% at $k = 8$). Its error also barely falls as more data is added (Figure 3), which is the signature of bias rather than variance. Because the ground-truth CPT columns are drawn independently, the true conditionals contain full interactions between parents, which an additive model cannot fit. Sharing parameters therefore helps only when the shared model is flexible enough to represent the true conditionals, and the hidden layer provides that flexibility.

Figure 2(b) shows that the networks do not save parameters. Their size grows only linearly with in-degree, because their input width is the sum of parent cardinalities, but the tuned networks still hold far more parameters than the tables they replace: `nn_256` holds 89 896 at $k = 10$, eight times the CPT's 11 263. What neural CPDs save is data, by sharing statistical strength across parent configurations.

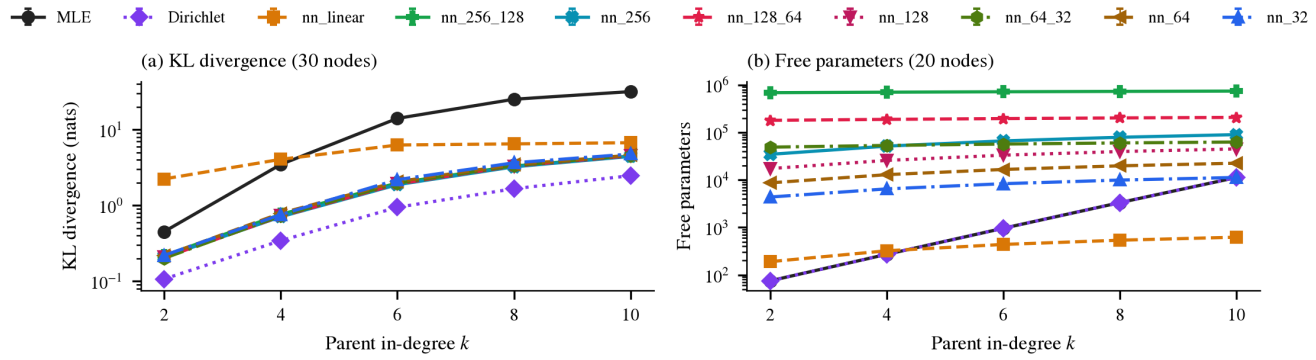


Figure 2: All estimators at $r = 2$ ($\alpha = 0.5$, mean over sample sizes and five DAGs; log-scaled y-axes). (a) Mean KL divergence against parent in-degree k for 30-node networks; error bars are ± 1 standard error across the five DAGs, most smaller than the markers. (b) Free parameters for a 20-node network.

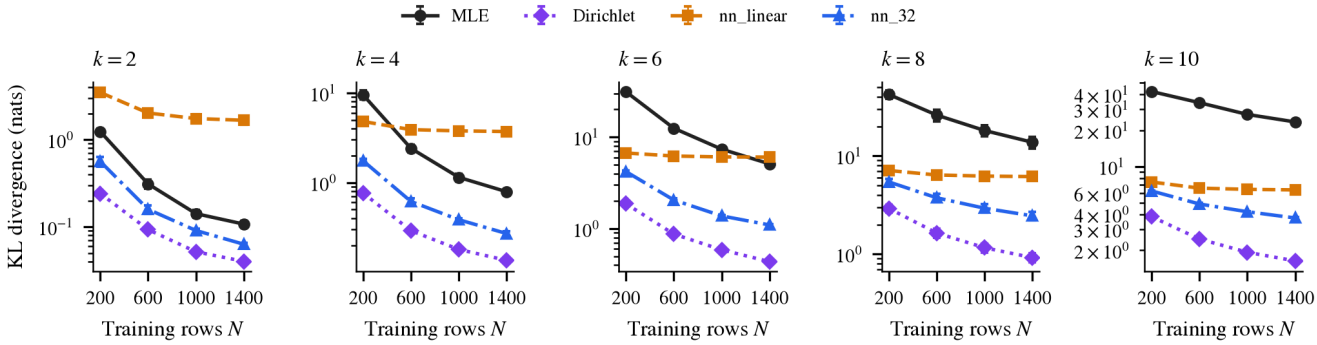


Figure 3: KL divergence against training rows at each in-degree ($r = 2$, 30-node networks, mean over five DAGs with error bars of ± 1 standard error across them, most smaller than the markers; log-scaled y-axis).

4.3. Neural CPDs Compared with MLE

Figure 3 shows how each estimator responds to more data. More rows help MLE, but at high in-degree it stays far worse than nn_32, while at $k = 2$ the two improve at a similar rate. Across the $r = 2$ grid, nn_32 beats MLE in 493 of 500 configurations (98.6%), rising from 93% at $k = 2$ to 100% at $k \geq 4$, and at $r = 4$ it beats MLE in all 300. The ordering of the estimators is the same at every network size; larger networks change only the scale of the KL divergence, which is summed over nodes (Supplementary Figures S4 and S5).

In-degree, cardinality and sample size all act on the estimators through the number of rows per parent configuration, $\rho = N/r^k$ (Eq. 2). Figure 4 plots the ratio of each tabular estimator's KL divergence to that of nn_32 against ρ, with one line per in-degree. If ρ captured all three factors, every line would fall on a single curve. Against MLE (panel a), whose ratios depend on the floor applied to its zero probabilities (Section 3.5), the $r = 2$ settings come close to this: the ratio falls from about 6–7 at $\rho \leq 1$ to about 2.3 at $\rho = 250$–350. Cardinality is not fully absorbed, since at the same ρ the ratio is higher at $r = 4$ (10.9 against 7.2 at $\rho = 0.78$). Below $\rho \approx 1$ the $r = 4$ ratio falls again, because when almost no configuration is observed both estimators stay close to their uninformed defaults. The advantage over MLE is therefore largest at roughly one training row per parent configuration. With much less data neither estimator can learn much, and with much more, MLE's tables are well estimated.

Against the Dirichlet estimator (panel b), the ratio is below 1 at every setting, from 0.41 to 0.62 at $r = 2$ and from 0.43 to 0.83 at $r = 4$. This is expected, because the matched-prior reference minimises expected KL divergence on these ground truths, so the ratio measures how far nn_32 remains from the best achievable estimate. The advantage does not come from knowing α: the Laplace estimator, which does not use it, has on average 7% higher KL divergence than the reference and still beats nn_32 in all 500 $r = 2$ configurations.

This accuracy comes at a computational cost. In the serial timing run on a 20-node network, MLE and the smoothed estimators fit in at most 0.3 s at $r = 2$, against 25–119 s for nn_32. At $r = 4$ and $k = 10$ the tables become very large, and MLE and the Dirichlet estimator take 58 s and 179 s, longer than nn_32 (15–25 s), although much of that time goes on building the tables in pgmpy. A network's cost grows with the number of training rows and nodes, and depends on in-degree mainly through early stopping, with training running longest at intermediate in-degree (about 355 epochs at $k = 6$, against about 210 at $k = 2$ and 245 at $k = 10$).

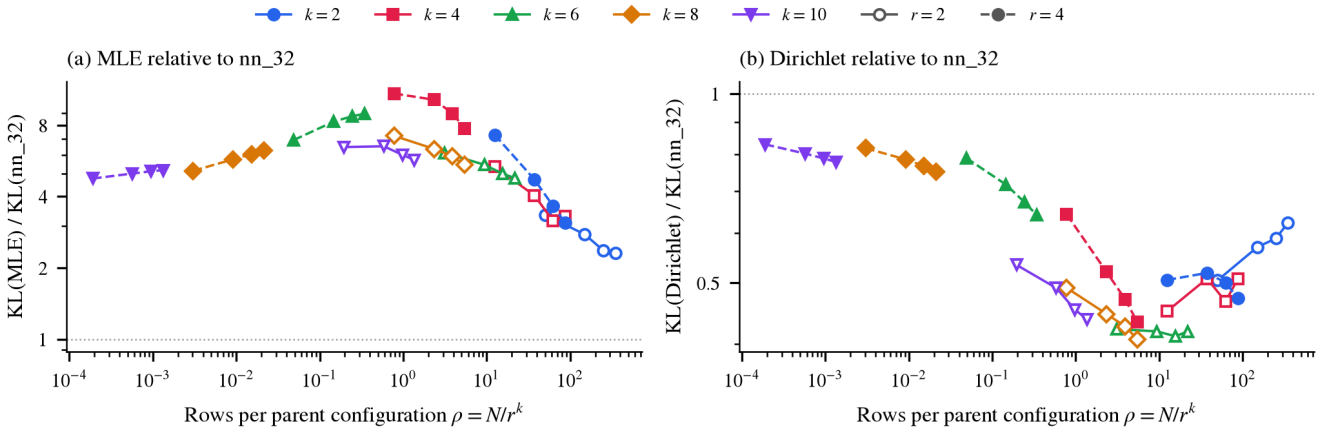


Figure 4: Ratio of (a) MLE's and (b) the Dirichlet estimator's KL divergence to that of nn_32, against rows per parent configuration $\rho = N/r^k$. Each line joins the four sample sizes ($N \in \{200, 600, 1000, 1400\}$) at one in-degree; open markers and solid lines are $r = 2$, filled markers and dashed lines $r = 4$. Each point is the mean per-DAG ratio over 15 DAGs (five each at 10, 20 and 30 nodes, $\alpha = 0.5$; log-scaled axes). Values above 1 favour nn_32. Panel (a) uses MLE's divergence with zero probabilities floored at 10^{-12}; a larger floor would lower its ratios.

4.4. Ground Truths with Shared Structure

Figure 5 repeats the comparison on the logistic ground truths, where data observed under one parent configuration is informative about others. The ordering now depends on in-degree. At $k = 2$ the tables are small and well covered, and the smoothed CPTs remain the most accurate. From $k = 6$ upwards, nn_32 is more accurate than every tabular estimator, including the Dirichlet estimator with its pseudocount tuned on validation data, in every configuration at both interaction levels. At $k = 10$ the best tabular estimator's KL divergence is 4.4 times that of nn_32 for additive CPDs and 2.4 times with interactions. A table estimates each configuration from its own rows, so it degrades once configurations are rarely observed, whereas a neural CPD can pool the rows of configurations that share a parent's effect.

The interaction level separates the two neural CPDs. With interactions, nn_32 beats the linear softmax in every configuration, because an additive model cannot represent the pairwise terms. For additive CPDs the linear softmax has the right functional form, and it beats every tabular estimator at $k \geq 8$, although, being untuned and limited to 200 epochs, it is often still improving when training stops. Together with the independent-column results, this indicates that the advantage of neural CPDs comes from structure shared across parent configurations: where that structure is absent, matched smoothing is optimal, and where it is present, neural CPDs win once the tables become sparse.

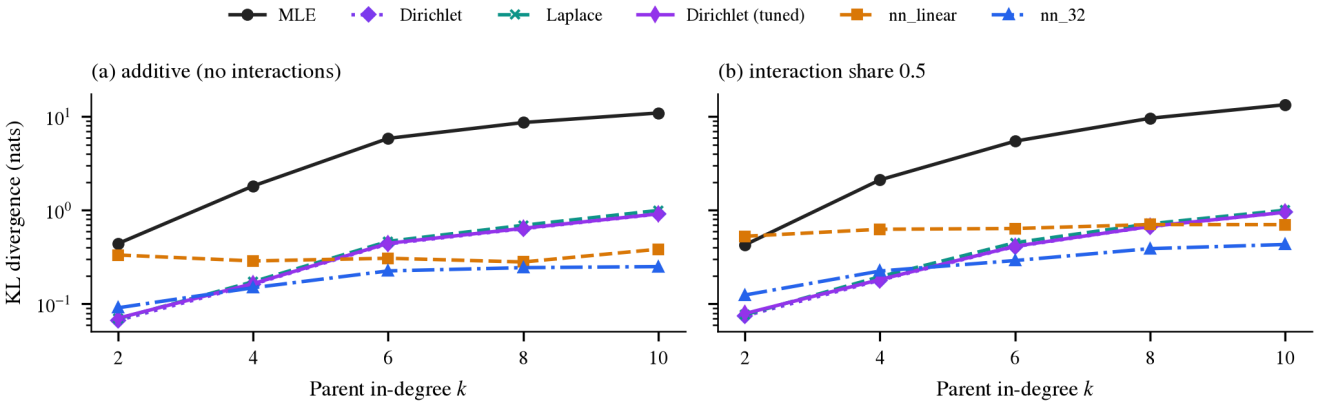


Figure 5: Logistic ground truths whose CPTs share structure across parent configurations ($r = 2$, 20-node networks, mean over $N \in \{200, 600, 1000, 1400\}$ and five DAGs; log-scaled y-axis): (a) purely additive CPDs, (b) half of the score variance from pairwise parent interactions. The tuned Dirichlet estimator has its pseudocount chosen on validation data.

4.5. Real Clinical Data

4.5.1. Predictive Performance

Figure 6 shows held-out log-likelihood averaged over all nine combinations of discretisation and DAG. Plain MLE is far below every other estimator at small N, because unseen child states receive near-zero probability. Both tuned networks outperform the tuned Dirichlet estimator in all 108 combinations of discretisation, DAG, N and fold (smallest margin 0.004 nats per row), and the untuned linear softmax does so in 105. The three networks perform almost identically: on average nn_32 is 0.004 nats per row better than the linear softmax and 0.019 better than nn_128_64.

Table 3 gives the gain of nn_32 over the Dirichlet estimator for each discretisation and sample size. It is largest at fine discretisation and small N, and falls as data grows, to 0.005 nats per row for L1 with the high-confidence graph at $N = 50\,000$. Finer discretisation and denser graphs both widen the gap, which reaches about 1.15 nats per row for L3

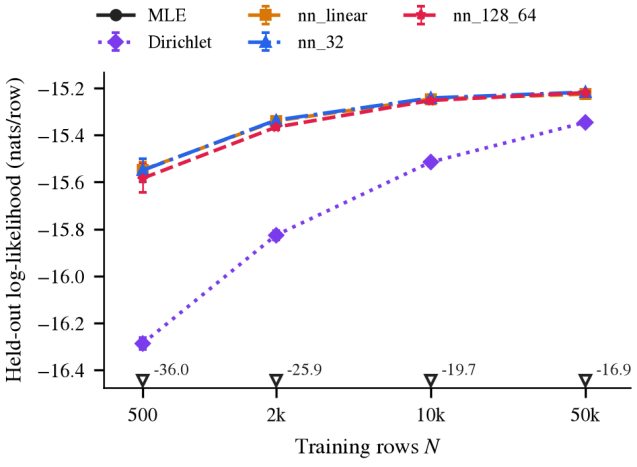


Figure 6: Held-out log-likelihood per row, mean over all nine settings and three folds, for each fold's tuned configurations; the linear softmax is untuned (higher is better; log-scaled x-axis).

Table 3
Gain in held-out log-likelihood of nn_32 over the tuned Dirichlet estimator (nats per row; mean ± s.e. of the paired differences over three DAG confidence levels and three folds). Every one of the 108 underlying differences is positive.

	$N = 500$	2 000	10 000	50 000
L1	0.35 ± 0.04	0.20 ± 0.03	0.08 ± 0.02	0.03 ± 0.01
L2	0.84 ± 0.05	0.56 ± 0.05	0.31 ± 0.05	0.14 ± 0.03
L3	1.03 ± 0.06	0.71 ± 0.05	0.43 ± 0.05	0.21 ± 0.04

with the low-confidence graph at $N = 500$ (Supplementary Figure S6). The pattern follows data coverage. With 50 000 rows, only 4% (L1), 16% (L2) and 28% (L3) of test rows meet a parent configuration never seen in training (Supplementary Figure S7), so most CPT entries are estimated from enough rows that sharing adds little.

Unlike in the synthetic experiments, the linear softmax is competitive with the hidden-layer networks on real data. Survey variables such as age, income and body mass index are ordinal and act on outcomes in largely monotone ways that an additive model can capture, whereas the synthetic CPT columns were drawn independently.

4.5.2. Diagnostic Accuracy

Held-out log-likelihood weighs all 22 variables equally, whereas a diagnostic model is judged on the query it is used for. Figure 7 shows how well each fitted network predicts diabetes from the other 21 variables. Both neural CPDs are better than the tuned Dirichlet estimator on all three measures in all 108 combinations. Their AUROC is already about 0.78 with 500 training rows and 0.81 with 50 000, whereas the Dirichlet estimator reaches 0.64 and 0.74. Its predictions are also poorly calibrated: in 93 of the 108 combinations its Brier score is worse than always predicting the 13.9% prevalence.

The reason is that diagnosis rests on the largest CPT in the network. In every expert DAG the diabetes node has the

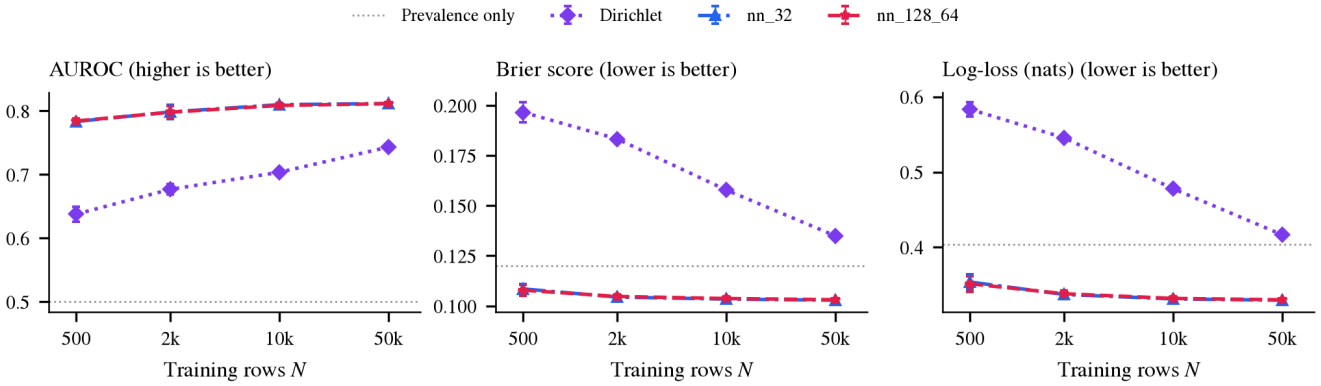


Figure 7: Diagnostic accuracy of the predicted probability of diabetes given all other variables, for each fold's tuned configurations: (a) AUROC, (b) Brier score and (c) log-loss, mean over all nine settings ± s.d. over three folds (log-scaled x-axis). The dotted line is the score of always predicting the 13.9% prevalence. MLE is omitted.

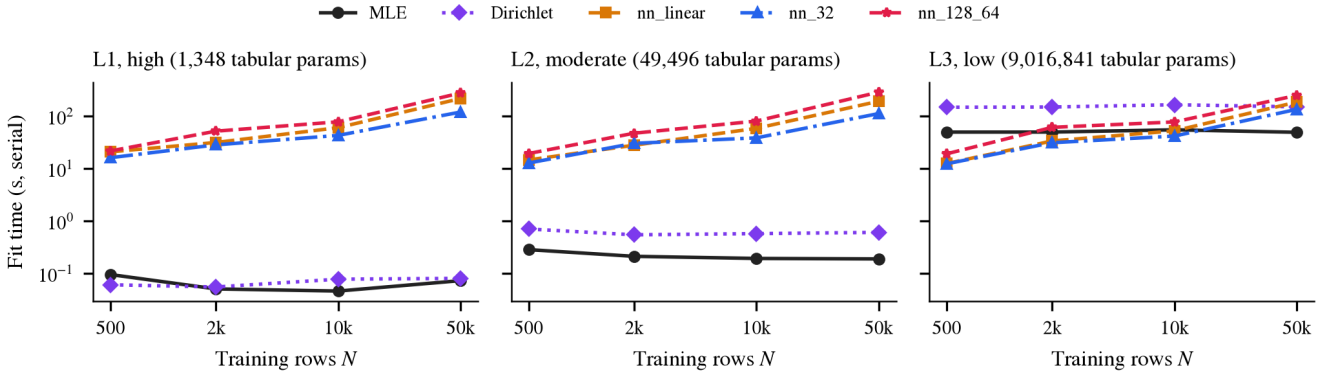


Figure 8: Fit time on the real dataset from a serial run on one fold, for three discretisation and DAG settings spanning the size of the tabular CPTs (log-scaled axes). The networks are timed with their learning rate before tuning (6×10^{-4}) and the same batch-size rule; the Dirichlet estimator with pseudocount 0.5.

most parents (7, 8 or 12), and so the most parent configurations, from 864 (L1, high confidence) to almost nine million (L3, low confidence). With 500 training rows, between 42% (L1) and 92% (L3) of test patients have a combination of risk factors that never occurs in training. For them a tabular estimator can only give its uninformed default, a 50% chance of diabetes, far from the true prevalence. A neural CPD instead predicts from the learned effect of each risk factor, so a patient with an unseen combination still receives a sensible probability.

4.5.3. Parameter and Training Cost

On real data the networks are far smaller than the tables they replace: at the densest setting, `nn_32` holds 13 364 parameters against the CPT's 9 016 841 (Supplementary Figure S8). They are not cheaper to fit, however. Figure 8 shows serial fit time for three settings spanning the size of the CPTs. The cost of the tabular estimators is set by the size of their tables rather than by N: they fit in under a second for the two smaller settings, but MLE and the Dirichlet estimator take about 50 s and 150 s for the 9-million-parameter CPT. The networks instead scale with N: `nn_32` takes 12–16 s at $N = 500$ and 111–134 s at $N = 50\,000$, and `nn_128_64` up to 285 s.

Where the networks are slower, they need far less data for the same accuracy. Table 4 gives the number of training rows the tuned Dirichlet estimator needs to reach the held-out log-likelihood of `nn_32`. Trained on 500 rows, `nn_32` is as accurate as the Dirichlet estimator trained on about 1 400 rows (L1, high confidence) to 34 000 rows (L3, low confidence), so the table needs between 3 and 67 times as much data. Trained on 2 000 rows, `nn_32` is not matched by the Dirichlet estimator even at 50 000 rows in three of the nine settings. The two costs are of different kinds: `nn_32` spends seconds of computation, while the table needs thousands of additional patient records to match it. Where records are the limiting resource, as in clinical data, this is a favourable exchange.

Table 4
Training rows the tuned Dirichlet estimator needs to match the mean held-out log-likelihood of `nn_32` trained on N rows (mean over three folds, interpolated on $\log N$ between 500, 2 000, 10 000 and 50 000 rows; rounded to three significant figures). >50 000: not matched within the grid. Each fold on its own gives values within a factor of 1.6 of these.

	DAG	$N = 500$	2 000	10 000
L1	high	1 380	6 650	36 400
	moderate	2 770	13 300	>50 000
	low	5 210	40 600	>50 000
L2	high	4 860	17 300	>50 000
	moderate	10 700	48 000	>50 000
	low	21 700	>50 000	>50 000
L3	high	6 120	29 400	>50 000
	moderate	15 500	>50 000	>50 000
	low	33 700	>50 000	>50 000

5. Conclusion and Future Work

Bayesian networks are a promising framework for medical diagnosis because their graphical structure makes probabilistic reasoning visible, but their standard estimators learn every parent configuration from its own observations and become unreliable when data is scarce relative to the number of configurations. This report replaced the conditional probability tables with neural CPDs while keeping the graph that makes the network interpretable.

On synthetic networks whose CPT columns are drawn independently, a smoothed table matched to the generating prior is the best possible estimator. Neural CPDs, which do not know that prior, stayed within about two and a half times its divergence and beat unsmoothed maximum likelihood in 793 of 800 configurations. When the CPTs share structure across configurations, neural CPDs beat every smoothed table, including one tuned on validation data, from in-degree 6, and a hidden layer is needed when parents interact. The clinical data show the same: neural CPDs outperformed a validation-tuned smoothed table in every setting and predicted diabetes far more accurately, because the diabetes node has the largest table in every expert graph. Fine clinical categories therefore need not be collapsed to keep tables estimable. The benefit is statistical rather than computational: a neural CPD can hold more parameters than the table it replaces, but the table needed 3 to 67 times as many records to match a small network trained on 500.

5.1. Limitations

The independent-column synthetic networks contain no structure shared across parent configurations and so favour smoothed tables by construction, and the structured ground truths cover only $r = 2$, 20-node networks and two interaction levels. Neither reproduces the noise or class imbalance of clinical data. Network settings were tuned on 20-node networks, and on real data the networks reuse the $r = 4$ settings with only the learning rate tuned. The selected learning rate and pseudocount are the largest values in their grids, and the real-data comparison rests on three folds. Finally, a neural CPD is less directly readable than a CPT: the graph remains interpretable, but each local distribution has to be queried rather than read off a table.

5.2. Future Work

The most natural extension is to hybrid networks. Many physiological measurements, such as blood pressure and glucose, are continuous, and discretising them loses information. Gaogle (2026) showed that neural CPDs work for Gaussian Bayesian networks, and combining the two approaches would let clinical variables be modelled in their natural form. Richer clinical data, such as critical care databases with many more variables and rarer conditions, would test neural CPDs in the regime of few rows per configuration where they were most valuable here. Monotonicity or sign constraints elicited from clinicians could encode accepted medical relationships in the networks directly. Finally, the diagnostic query here observes every other variable and so needs only the Markov blanket of the diagnosis node; queries with partial evidence require inference through the whole network, for example by generating a neural CPD's table on demand, and evaluating them would test whether the gains carry through to clinical decision support.

Ethics and Data Use

This project involves no human participants. The real-data experiments use the CDC Diabetes Health Indicators dataset (Centers for Disease Control and Prevention, 2017), an anonymised, publicly available extract of the 2015 Behavioral Risk Factor Surveillance System survey collected by the US Centers for Disease Control and Prevention and redistributed by the UCI Machine Learning Repository from a public-domain release (Creative Commons CC0 1.0). It contains no identifiers. The synthetic data are sampled from randomly generated networks over abstract variables. The project was classified as no risk and received an ethics waiver from the School of Computer Science and Applied Mathematics at the University of the Witwatersrand, under the University's Human Research Ethics Committee (Non-Medical) (waiver number WCSAM-2026-48).

Code and Data Availability

The code for every experiment, the results files, and the notebooks that produce each figure and table in this report are available at `https://github.com/kay-asp/Neural-Discrete-Bayesian-Ne`
The CDC Diabetes Health Indicators data are available from the UCI Machine Learning Repository (Centers for Disease Control and Prevention, 2017), and the expert-elicited DAGs from Zahoor et al. (2025); copies of both are included in the repository.

Supplementary Material

The supplementary material contains the architecture-selection results and the settings of every estimator, the

moderate-confidence expert DAG, the synthetic results for each network size, and the real-data results, unseen-configuration rates and parameter counts for each discretisation and DAG.

References

Bengio, S., Bengio, Y., 2000. Taking on the curse of dimensionality in joint distributions using neural networks. IEEE Transactions on Neural Networks 11, 550–557. doi:10.1109/72.846725.

Centers for Disease Control and Prevention, 2017. CDC diabetes health indicators. UCI Machine Learning Repository, dataset 891. URL: https://archive.ics.uci.edu/dataset/891, doi:10.24432/C53919.

Di, R.h., Wang, P., He, C., Guo, Z.g., 2021. Constrained adjusted maximum a posteriori estimation of bayesian network parameters. Entropy doi:10.3390/E23101283.

Friedman, N., Goldszmidt, M., 1996. Learning Bayesian networks with local structure, in: Proceedings of the Twelfth Conference on Uncertainty in Artificial Intelligence, pp. 252–262.

Gaogle, A., 2026. Using Neural Networks to Estimate the Parameters of Gaussian Bayesian Networks. Master's thesis. University of the Witwatersrand. Johannesburg, South Africa. Unpublished Masters dissertation.

Goodfellow, I., Bengio, Y., Courville, A., 2016. Deep Learning. MIT Press. URL: https://www.deeplearningbook.org.

Heckerman, D., Geiger, D., Chickering, D.M., 1995. Learning Bayesian networks: The combination of knowledge and statistical data. Machine Learning 20, 197–243. doi:10.1007/BF00994016.

Hulsen, T., 2023. Explainable artificial intelligence (xai): Concepts and challenges in healthcare. Applied Informatics doi:10.3390/AI4030034.

Ke, N.R., Bilaniuk, O., Goyal, A., Bauer, S., Larochelle, H., Schölkopf, B., Mozer, M.C., Pal, C., Bengio, Y., 2019. Learning neural causal models from unknown interventions. arXiv:1910.01075.

Kingma, D.P., Ba, J., 2015. Adam: A method for stochastic optimization, in: International Conference on Learning Representations.

Koller, D., Friedman, N., 2009. Probabilistic graphical models: principles and techniques. MIT press.

Koller, D., Friedman, N., Getoor, L., Taskar, B., 2007. Graphical models in a nutshell, in: Getoor, L., Taskar, B. (Eds.), Introduction to Statistical Relational Learning. MIT Press, Cambridge, MA, pp. 13–55.

Kyrimi, E., McLachlan, S., Dube, K., Neves, M.R., Fahmi, A., Fenton, N., 2021. A comprehensive scoping review of bayesian networks in healthcare: Past, present and future. Artificial Intelligence in Medicine doi:10.1016/J.ARTMED.2021.102108.

Larochelle, H., Murray, I., 2011. The neural autoregressive distribution estimator, in: Proceedings of the Fourteenth International Conference on Artificial Intelligence and Statistics, pp. 29–37.

Loshchilov, I., Hutter, F., 2019. Decoupled weight decay regularization, in: International Conference on Learning Representations.

Nafar, A., Venable, K.B., Cui, Z., Kordjamshidi, P., 2026. Extracting probabilistic knowledge from large language models for Bayesian network parameterization. Transactions on Machine Learning Research ArXiv:2505.15918.

Neal, R.M., 1992. Connectionist learning of belief networks. Artificial Intelligence 56, 71–113.

Niculescu, R.S., Mitchell, T.M., Rao, R.B., 2006. Bayesian network learning with parameter constraints. Journal of Machine Learning Research 7, 1357–1383.

Oniśko, A., Drużdżel, M.J., 2013. Impact of precision of bayesian network parameters on accuracy of medical diagnostic systems. Artificial Intelligence in Medicine doi:10.1016/J.ARTMED.2013.01.004.

Oniśko, A., Drużdżel, M.J., Wasyluk, H., 2001. Learning bayesian network parameters from small data sets: application of noisy-or gates. International Journal of Approximate Reasoning doi:10.1016/S0888-613X(01)00039-1.

Petitjean, F., Buntine, W., Webb, G.I., Zaidi, N., 2018. Accurate parameter estimation for Bayesian network classifiers using hierarchical Dirichlet processes. Machine Learning 107, 1303–1331. doi:10.1007/s10994-018-5718-0.

Rijmen, F., 2008. Bayesian networks with a logistic regression model for the conditional probabilities. International Journal of Approximate Reasoning 48, 659–666. doi:10.1016/j.ijar.2008.01.001.

Rudin, C., 2019. Stop explaining black box machine learning models for high stakes decisions and use interpretable models instead. Nature Machine Intelligence doi:10.1038/S42256-019-0048-X.

Zahoor, S., Constantinou, A.C., Curtis, T.M., Hasanuzzaman, M., 2025. Investigating the validity of structure learning algorithms in identifying risk factors for intervention in patients with diabetes. Knowledge-Based Systems 329, 114382. doi:10.1016/j.knosys.2025.114382.

Zhong, L., Yang, F., Sun, S., Wang, L., Yu, H., Nie, X., Liu, A., Xu, N., Zhang, L., Zhang, M., Qi, Y., Ji, H., Liu, G., Zhao, H., Jiang, Y., Li, J., Song, C., Yu, X., Yang, L., Yu, J., Feng, H., Guo, X., Yang, F., Xue, F., 2024. Predicting lung cancer survival prognosis based on the conditional survival Bayesian network. BMC Medical Research Methodology 24, 16. doi:10.1186/s12874-023-02043-y.